Institut für Parallele und Verteilte Systeme

Universität Stuttgart
Universitätsstraße 38
D–70569 Stuttgart

Masterarbeit

# Connecting MetaConfigurator to the Semantic Web: RDF Authoring, Ontology Integration, and Knowledge Graph Support.

Mahdi Jafarkhani

| | |
|---|---|
| Studiengang: | Computer Science |
| Prüfer/in: | Jun. Prof. Dr. rer.nat. Benjamin Uekermann |
| Betreuer/in: | Felix Neubauer, M.Sc. |
| Beginn am: | October 15, 2025 |
| Beendet am: | April 15, 2026 |

## Abstract

Scientific workflows increasingly produce structured JavaScript Object Notation (JSON) data that is easy to exchange but difficult to interpret semantically across systems. While JSON Schema supports structural validation, it does not provide native semantics for Linked Data integration. This thesis addresses that gap by extending MetaConfigurator, a schema-driven web application for structured data authoring, with Semantic Web capabilities.

The core contribution is an integrated Resource Description Framework (RDF) Authoring Panel that supports end-to-end semantic data workflows in a single user interface (UI). The extension enables transformation of JSON documents into JavaScript Object Notation for Linked Data (JSON-LD) using RDF Mapping Language (RML), direct authoring and editing of semantic structures, SPARQL Protocol and RDF Query Language (SPARQL)-based querying, and interactive knowledge graph visualization. In addition, artificial intelligence (AI)-assisted generation of SPARQL queries and RML mappings from user hints is introduced to reduce the entry barrier for non-expert users.

The result is a unified environment that bridges schema-driven configuration editing and Semantic Web technologies. By combining validation, transformation, querying, and visualization in one workflow, the approach improves interoperability of scientific experiment data and supports more accessible creation and exploration of semantically enriched knowledge graphs.

## Kurzfassung

Wissenschaftliche Workflows erzeugen zunehmend strukturierte JSON-Daten, die sich zwar leicht austauschen lassen, deren semantische Interpretation über Systemgrenzen hinweg jedoch schwierig ist. JSON Schema ermöglicht strukturelle Validierung, bietet aber keine nativen Semantiken für die Integration in Linked-Data-Infrastrukturen. Diese Arbeit schließt diese Lücke, indem MetaConfigurator, eine schema-getriebene Webanwendung zur Bearbeitung strukturierter Daten, um Semantic-Web-Funktionen erweitert wird.

Der zentrale Beitrag ist ein integriertes RDF-Authoring-Panel, das durchgängige semantische Daten-Workflows in einer einzigen Benutzeroberfläche unterstützt. Die Erweiterung umfasst die Transformation von JSON-Dokumenten nach JSON-LD mittels RML, die direkte Erstellung und Bearbeitung semantischer Strukturen, SPARQL-basierte Abfragen sowie eine interaktive Visualisierung von Wissensgraphen. Ergänzend wird eine KI-gestützte Generierung von SPARQL-Abfragen und RML-Mappings aus Nutzerhinweisen eingeführt, um die Einstiegshürde für Nicht-Expert:innen zu senken.

Als Ergebnis entsteht eine einheitliche Umgebung, die schema-getriebene Konfigurationsbearbeitung mit Semantic-Web-Technologien verbindet. Durch die Kombination von Validierung, Transformation, Abfrage und Visualisierung in einem Workflow verbessert der Ansatz die Interoperabilität wissenschaftlicher Versuchsdaten und erleichtert die Erstellung sowie Exploration semantisch angereicherter Wissensgraphen.

# Contents

# List of Figures

# List of Tables

# Abbreviations

| Notation | Description | Page List |
|---|---|---|
| AI | artificial intelligence | 3, 25, 27, 29, 30 |
| CEDAR | Center for Expanded Data Annotation and Retrieval | 28 |
| CSV | comma-separated values | 20 |
| ETL | extract, transform, load | 28 |
| GUI | graphical user interface | 23 |
| IRI | Internationalized Resource Identifier | 19, 20, 26, 27 |
| JSON | JavaScript Object Notation | 3, 4, 13, 14, 16, 17, 19–21, 23, 26–28, 30 |
| JSON-LD | JavaScript Object Notation for Linked Data | 3, 4, 13, 16–21, 26–28, 30 |
| JSONPath | JSON Path | 21 |
| OWL | Web Ontology Language | 28, 29 |
| QUDT | Quantities, Units, Dimensions, and Data Types | 17, 18 |
| R2RML | RDB to RDF Mapping Language | 20, 30 |

| Notation | Description | Page List |
|---|---|---|
| RDF | Resource Description Framework | 3, 4, 16–22, 26–30 |
| RDFLib | RDFLib | 28 |
| RML | RDF Mapping Language | 3, 4, 13, 20, 21, 26, 27, 30 |
| SHACL | Shapes Constraint Language | 17, 28 |
| SPARQL | SPARQL Protocol and RDF Query Language | 3, 4, 17, 18, 26–30 |
| SQL | Structured Query Language | 21 |
| UI | user interface | 3, 23 |
| URI | Uniform Resource Identifier | 24 |
| W3C | World Wide Web Consortium | 17, 20 |
| XML | Extensible Markup Language | 20 |
| XPath | XML Path Language | 21 |
| YAML | YAML Ain't Markup Language | 23 |

# 1 Background and Related Work

The increasing adoption of computational experiments and simulation workflows in scientific research has resulted in large volumes of structured data describing parameters, metrics, and results. Ensuring that such data can be validated, exchanged, and semantically interpreted requires standardized mechanisms for describing structure and meaning. This chapter introduces three key technologies that enable this process: JSON for structured data representation, JSON Schema for structural validation, and semantic mappings using RML to produce semantically enriched JSON-LD representations. The discussion is illustrated using a running example from a simulation experiment where the element size parameter influences a computed stress metric.

## 1.1 JSON and JSON Schema

### 1.1.1 JSON as a Data Representation Format

JSON is a lightweight data-interchange format widely used for representing structured data in web services, scientific workflows, and configuration systems. JSON encodes data as attribute-value pairs and ordered lists, making it human-readable and easy to parse by machines. Due to its simplicity and broad ecosystem support, JSON has become a de facto standard for structured data exchange across many domains [Bra17].

In computational science workflows, JSON files often capture experiment parameters and resulting metrics. The following example illustrates a minimal JSON document describing the input parameter `element-size` and the resulting metric `max_von_mises_stress_nodes`.

```
{
    "parameters": {
        "element-size": {
            "value": 0.025,
            "unit": "m"
        }
    },
    "metrics": {
        "max_von_mises_stress": 299791507.5586333
```

```
10      }
11  }
```

Listing 1.1: Example JSON representation of simulation input parameters and results

In Listing 1.1, the `parameters.element-size` object stores the input value (`0.025`) and its unit (`m`), while `metrics.max_von_mises_stress` captures the computed simulation output. In our benchmark example, `element-size` represents a meshing property that controls mesh density for $h$-refinement, i.e., the characteristic size of finite elements. The `max_von_mises_stress` value is a scalar result metric defined as the maximum von Mises stress over the simulated domain.

While JSON provides a convenient serialization format, it does not inherently enforce structural constraints. Therefore, additional mechanisms are required to validate that JSON documents conform to an expected structure.

### 1.1.2 JSON Schema for Data Validation

JSON Schema provides a standardized mechanism for describing the expected structure and constraints of JSON documents. A JSON Schema defines properties, data types, required attributes, and validation rules for JSON instances. This allows automated validation of data before further processing or transformation. The JSON Schema specification defines a vocabulary for describing JSON documents and their validation rules [WAHD20].

Listing 1.2 shows a JSON Schema corresponding to the example JSON document in Listing 1.1. The schema ensures that the JSON document contains the expected parameter and metric fields and enforces the correct numeric data types.

```
1  {
2    "$schema": "https://json-schema.org/draft/2020-12/schema",
3    "type": "object",
4    "properties": {
5      "parameters": {
6        "type": "object",
7        "properties": {
8          "element-size": {
9            "type": "object",
10           "properties": {
11             "value": {
12               "type": "number"
13             },
14             "unit": {
15               "type": "string"
```

```
16              }
17            },
18            "required": ["value", "unit"]
19          }
20        },
21        "required": ["element-size"]
22      },
23      "metrics": {
24        "type": "object",
25        "properties": {
26          "max_von_mises_stress": {
27            "type": "number"
28          }
29        },
30        "required": ["max_von_mises_stress"]
31      }
32    },
33    "required": ["parameters", "metrics"]
34  }
```

Listing 1.2: JSON Schema validating the simulation result JSON document

The schema in Listing 1.2 directly mirrors the example instance in Listing 1.1: it requires both top-level objects (`parameters`, `metrics`), enforces the nested `element-size.value/unit` fields, and constrains the stress metric to a numeric type.

Using JSON Schema enables consistent validation of experiment data and ensures that all required values are present before transformation into other representations.

However, JSON Schema focuses primarily on structural validation and does not provide semantic meaning or interoperability with knowledge graph technologies. To address this limitation, semantic mapping techniques can be applied.

## 1.2 Semantic Web

### 1.2.1 Ontology: Modeling Knowledge Domains

Ontologies provide a formal specification of concepts and relationships within a domain of knowledge. One of the most widely cited and influential definitions is provided by Gruber:

> "An ontology is an explicit specification of a shared conceptualization." [Gru93]

In this definition, the key terms can be interpreted as follows:

- **Explicit**: The concepts and relations are stated clearly and unambiguously, rather than being left implicit in code or documentation.
- **Specification**: The concepts are described in a formal, structured model (e.g., classes, properties, constraints, and axioms) that can be processed consistently by both humans and machines.
- **Shared conceptualization**: The model captures a common understanding of a domain agreed upon by a community, allowing data created by different people or systems to be interpreted in the same way.

Later work further emphasizes that ontologies should be formal and shared in order to support machine interpretation and interoperability across communities [Gua98; SBF98]. They enable shared understanding among humans and machines by defining classes, properties, and axioms that describe entities and their interrelations.

In the context of scientific experiments, ontologies can define experiment parameters, measurement units, procedures, and results. For example, an ontology might define a class `PropertyValue` representing any measured property, a class `Method` representing experimental procedures, and relationships such as `hasParameter` and `investigates` to link methods to parameters and observed metrics.

Ontologies serve as the backbone for semantic data integration and reasoning. They allow data from heterogeneous sources to be interpreted consistently, facilitate interoperability, and enable advanced queries across datasets using formal reasoning engines.

### 1.2.2 Semantic Web Concepts

The Semantic Web extends the World Wide Web by enabling data to be represented in a machine-readable, semantically rich form. Data is described using standards such as RDF, which encodes information as triples consisting of subject, predicate, and object. This allows heterogeneous data to be interlinked and queried using semantic technologies [BHL01].

JSON-LD is one of the key technologies bridging conventional JSON data and the Semantic Web. By adding semantic annotations, JSON-LD enables JSON documents to be interpreted as RDF graphs. For instance, in the example simulation data (Listing 1.6), the simulation parameter `element-size` is represented as a `schema:PropertyValue` class.

```
1 {
2     "@id": "local:variable_element_size",
3     "@type": "schema:PropertyValue",
4     "rdfs:label": "element-size",
```

```
    "schema:unitCode": {
        "@id": "unit:M"
    },
    "schema:value": 0.025
}
```

Listing 1.3: Example of semantic annotation of a parameter with QUDT unit

Here, the `element-size` is measured in meters, and is linked to the `unit:M` as its `schema:unitCode`. This allows applications and knowledge graph systems to unambiguously interpret the measurement unit and integrate it with other semantically annotated datasets. Similarly, experiment methods and metrics are linked to well-defined ontology classes such as `m4i:Method` and `schema:PropertyValue`.

Semantic Web technologies thus provide a framework for:

- Interoperable data representation across different systems and domains.
- Linking and reasoning over experimental data using ontologies.
- Querying and integrating datasets through standard languages like SPARQL.

By grounding JSON data in ontologies and RDF through JSON-LD, this simulation result becomes part of the Semantic Web, enabling more advanced analysis, automated reasoning, and integration with other scientific knowledge graphs.

### 1.2.3 SHACL: Shapes Constraint Language

The Shapes Constraint Language (SHACL) is a World Wide Web Consortium (W3C) standard for validating RDF graphs against a set of structural and semantic constraints [W3C17]. SHACL allows ontology-based validation of RDF data by defining ”shapes”that describe the expected properties, data types, and cardinality of RDF nodes.

For example, consider the simulation parameter `element-size` in the JSON-LD example (Listing 1.6). Using SHACL, we can define a shape that enforces that every `schema:PropertyValue` representing an experiment parameter must have a numeric `schema:value` and a `schema:unitCode` pointing to a valid unit from the Quantities, Units, Dimensions, and Data Types (QUDT) ontology:

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix schema: <https://schema.org/> .
@prefix unit: <http://qudt.org/vocab/unit/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .

```

```
ex:ParameterShape
    a sh:NodeShape ;
    sh:targetClass schema:PropertyValue ;
    sh:property [
        sh:path schema:value ;
        sh:datatype xsd:decimal ;
        sh:minCount 1 ;
    ] ;
    sh:property [
        sh:path schema:unitCode ;
        sh:nodeKind sh:IRI ;
        sh:minCount 1 ;
    ] .
```

Listing 1.4: SHACL shape for validating simulation parameters

By applying this shape, RDF data can be automatically checked for compliance with the expected structure and semantics, preventing errors or inconsistencies before integration into a knowledge graph.

### 1.2.4 SPARQL: Querying Semantic Data

SPARQL is the standard query language for RDF datasets [PS13]. SPARQL allows retrieval, filtering, and aggregation of data stored in knowledge graphs. Queries are expressed in terms of triples, similar to RDF statements.

For example, using the JSON-LD simulation data, we might want to retrieve all experiment parameters along with their values and units. A SPARQL query could be written as:

```
PREFIX schema: <https://schema.org/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?parameter ?value ?unit
WHERE {
    ?param a schema:PropertyValue ;
           rdfs:label ?parameter ;
           schema:value ?value ;
           schema:unitCode ?unit .
}
```

Listing 1.5: SPARQL query retrieving parameter values and units

This query returns the `element-size` parameter with its numeric value and QUDT unit (`unit:M`), demonstrating how semantic annotations enable precise data access. SPARQL supports more

complex operations such as joins, filters, and reasoning over ontologies, making it a key tool for exploring and analyzing semantically enriched simulation data.

### 1.2.5 Semantic Data Representation using JSON-LD

JSON-LD extends JSON by introducing semantic annotations that allow data to be interpreted as linked data within RDF [SKL14a]. JSON-LD integrates JSON with established Semantic Web standards by providing mechanisms for defining contexts, identifiers, and relationships between entities.

A typical JSON-LD document has two key structural components:

- **@context:** This section defines the mapping between JSON keys and ontology International-alized Resource Identifiers (IRIs) or vocabularies. It provides semantic meaning to each field and enables JSON documents to be interpreted as RDF triples.
- **@graph:** This array contains the actual data entities, each represented as a node with an `@id`, a `@type`, and associated properties. Relationships between entities are expressed using references to other `@id` values, forming a connected graph structure.

Listing 1.6 shows a JSON-LD representation corresponding to the example simulation result. The `@context` defines prefixes such as `schema` and `unit`, while the `@graph` contains nodes representing the simulation run, the element-size parameter, and the stress metric.

```
{
    "@context": {
        "local": "https://www.example.org/",
        "schema": "https://schema.org/",
        "m4i": "http://w3id.org/nfdi4ing/metadata4ing#",
        "rdfs": "http://www.w3.org/2000/01/rdf-schema#",
        "unit": "http://qudt.org/vocab/unit/"
    },
    "@graph": [
        {
            "@id": "local:run_fenics_simulation",
            "@type": "m4i:Method",
            "m4i:hasParameter": [{"@id": "local:variable_element_size"}],
            "m4i:investigates": [{"@id": "local:variable_max_von_mises_stress"}]
        },
        {
            "@id": "local:variable_element_size",
            "@type": "schema:PropertyValue",
```

```
19              "rdfs:label": "element-size",
20              "schema:unitCode": {"@id": "unit:M"},
21              "schema:value": 0.025
22          },
23          {
24              "@id": "local:variable_max_von_mises_stress",
25              "@type": "schema:PropertyValue",
26              "rdfs:label": "max_von_mises_stress",
27              "schema:value": 299791507.5586333
28          }
29      ]
30  }
```

Listing 1.6: JSON-LD representation of the simulation result

This structure allows each entity to be unambiguously identified, linked, and queried using RDF-aware tools. By separating semantic definitions (`@context`) from the data graph (`@graph`), JSON-LD documents support both human readability and machine interoperability, enabling integration with knowledge graphs and other Semantic Web infrastructures.

### 1.2.6 RML for Mapping JSON to RDF

RML is a declarative language for transforming heterogeneous data sources into RDF representations. RML extends the W3C-standardized RDB to RDF Mapping Language (R2RML) language and supports mappings from JSON, Extensible Markup Language (XML), comma-separated values (CSV), and relational databases into RDF datasets [DVC+14].

Formally, an RDF triple is a 3-tuple:

$$(s, p, o) \in (U \cup B) \times U \times (U \cup B \cup L)$$

where:

- $U$ is the set of all IRIs,
- $B$ is the set of blank nodes (anonymous resources),
- $L$ is the set of RDF literals (e.g., strings, numbers, dates),
- $s \in U \cup B$ is the subject,
- $p \in U$ is the predicate (also called property),
- $o \in U \cup B \cup L$ is the object.

An RDF graph $G$ is then defined as a finite set of such triples:

$$G = \{(s_i, p_i, o_i) \mid i = 1, \ldots, n\}, \quad s_i \in U \cup B,\ p_i \in U,\ o_i \in U \cup B \cup L$$

RML mappings are themselves RDF graphs specifying how elements from a source data structure are transformed into RDF triples. Each mapping defines:

- **Logical source:** the data source and reference formulation (e.g., JSON Path (JSONPath), XML Path Language (XPath), Structured Query Language (SQL) query),
- **Subject map:** rules to generate the *subject* of triples,
- **Predicate-object maps:** rules to generate the *predicate* and *object* for triples.

Listing 1.7 shows an example RML mapping that converts the JSON simulation output from Listing 1.1 into the JSON-LD structure presented in Listing 1.6.

```
@prefix rml: <http://semweb.mmlab.be/ns/rml#>.
@prefix rr: <http://www.w3.org/ns/r2rml#>.
@prefix ql: <http://semweb.mmlab.be/ns/ql#>.
@prefix schema: <https://schema.org/>.
@prefix m4i: <http://w3id.org/nfdi4ing/metadata4ing#>.
@prefix unit: <http://qudt.org/vocab/unit/>.
@prefix local: <https://www.example.org/>.

<#MethodMapping>
    rml:logicalSource [
        rml:source "parameter.json" ;
        rml:referenceFormulation ql:JSONPath ;
        rml:iterator "$"
    ];
    rr:subjectMap [
        rr:constant local:run_fenics_simulation ;
        rr:class m4i:Method ;
    ];
    rr:predicateObjectMap [
        rr:predicate m4i:hasParameter ;
        rr:objectMap [ rr:constant local:variable_element_size ]
    ];
    rr:predicateObjectMap [
        rr:predicate m4i:investigates ;
        rr:objectMap [ rr:constant local:variable_max_von_mises_stress ]
    ].

<#ElementSizeMapping>
    rml:logicalSource [
```

```
        rml:source "parameter.json" ;
        rml:referenceFormulation ql:JSONPath ;
        rml:iterator "$.parameters.element-size"
    ];
    rr:subjectMap [
        rr:constant local:variable_element_size ;
        rr:class schema:PropertyValue ;
    ];
    rr:predicateObjectMap [
        rr:predicate schema:value ;
        rr:objectMap [ rml:reference "value" ]
    ];
    rr:predicateObjectMap [
        rr:predicate schema:unitCode ;
        rr:objectMap [ rr:constant unit:M ]
    ].

<#StressMetricMapping>
    rml:logicalSource [
        rml:source "simulation.json" ;
        rml:referenceFormulation ql:JSONPath ;
        rml:iterator "$.metrics"
    ];
    rr:subjectMap [
        rr:constant local:variable_max_von_mises_stress ;
        rr:class schema:PropertyValue ;
    ];
    rr:predicateObjectMap [
        rr:predicate schema:value ;
        rr:objectMap [ rml:reference "max_von_mises_stress" ]
    ].
```

Listing 1.7: RML mapping from JSON simulation output to RDF/JSON-LD

As one concrete example, `<#ElementSizeMapping>` reads the nested JSON object at `$.parameters.element-size`, creates the RDF resource `local:variable_element_size` of type `schema:PropertyValue`, then maps the JSON field `value` to `schema:value` and assigns the constant unit `unit:M` via `schema:unitCode`.

Through this mapping process, structured simulation outputs are automatically converted into RDF triples forming a semantically enriched knowledge graph.

## 1.3 MetaConfigurator for Schema-Driven Data Modeling

The technologies discussed in the previous sections enable structured, validated, and semantically enriched data representations. However, creating and editing such structured data manually can be challenging, particularly for users who are not familiar with textual formats such as JSON or with schema-based validation mechanisms. To address this challenge, tools that support schema-driven data modeling and editing are required.

MetaConfigurator[1] is an open-source web application designed to simplify the creation, editing, and management of structured data and schemas [NBX+24]. The tool generates graphical user interfaces (GUIs) dynamically based on a provided schema, allowing users to interact with structured data through intuitive forms rather than manually editing raw text files. By leveraging JSON Schema as the underlying model description, MetaConfigurator enables consistent data validation and guided data entry.

MetaConfigurator follows a schema-driven approach: the structure of the UI is derived directly from the schema that describes the data model. This design allows the same tool to be applied across different domains without the need to implement domain-specific UIs. The application runs entirely in the user's browser and supports multiple structured data formats such as JSON and YAML Ain't Markup Language (YAML) [NBX+24].

### 1.3.1 Core Features

MetaConfigurator provides several features that facilitate the creation and maintenance of structured data models:

- **Visual Schema Editor:** Users can create and modify JSON schemas using a graphical interface, enabling intuitive construction of data models without manually writing schema definitions. Figure 1.1 depicts an example of schema editing in MetaConfigurator, showing the JSON Schema from 1.2. The left panel shows the raw JSON Schema, the middle panel provides a diagram view for editing, and the right panel offers a structured form for schema modification.
- **Schema-Driven Form Generation:** Based on a given JSON Schema, MetaConfigurator automatically generates a graphical form for editing instance data. This ensures that all entered data conforms to the defined structure and constraints.
- **Integrated Text and GUI Editing:** The tool combines a traditional text editor with a graphical editor, allowing users to switch between raw JSON editing and structured form-based editing within the same interface. Figure 1.2 depicts an example of data editing in

[1] https://www.metaconfigurator.org

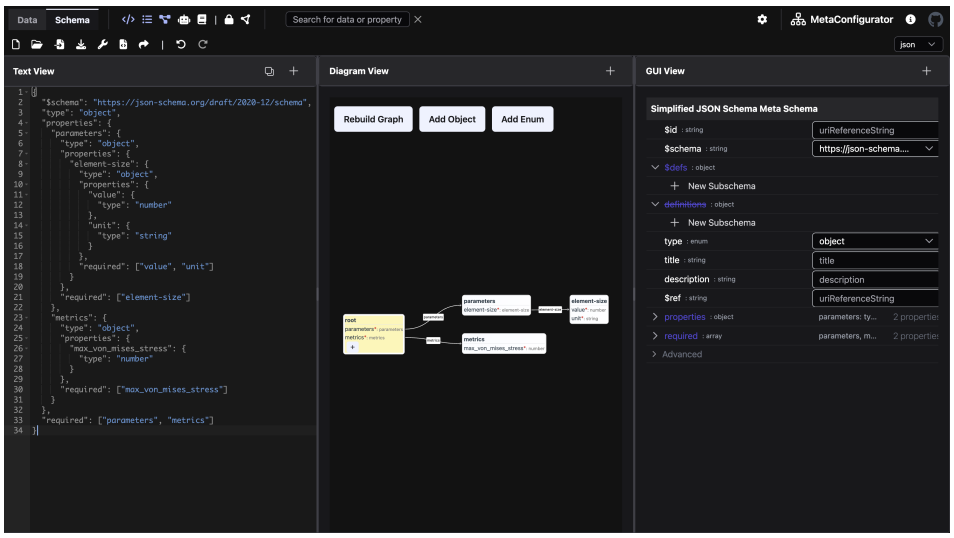


Figure 1.1: A snapshot of MetaConfigurator's schema editing interface.

MetaConfigurator, showing the JSON instance from 1.3 that conforms to the JSON Schema in 1.2. The left panel displays the raw JSON data, while the right panel offers a structured form for editing.

- **Schema Validation Support:** During data editing, MetaConfigurator validates instance data against the provided JSON Schema, ensuring structural correctness and preventing invalid data entries.
- **Schema Visualization and Navigation:** Complex schemas can be explored using a tree-like or diagram-based representation, helping users understand hierarchical structures and relationships between properties.
- **Limited Ontology Integration:** While the tool offers a JSON-LD option that introduces `@context` and `@id` fields, its overall support for ontologies remains very limited. It allows Uniform Resource Identifier (URI)-based references and provides auto-completion to reference ontology concepts when creating structured data models, but deeper integration is not available.

These capabilities significantly reduce the effort required to design and maintain structured data models and improve accessibility for users who may not be familiar with schema languages.

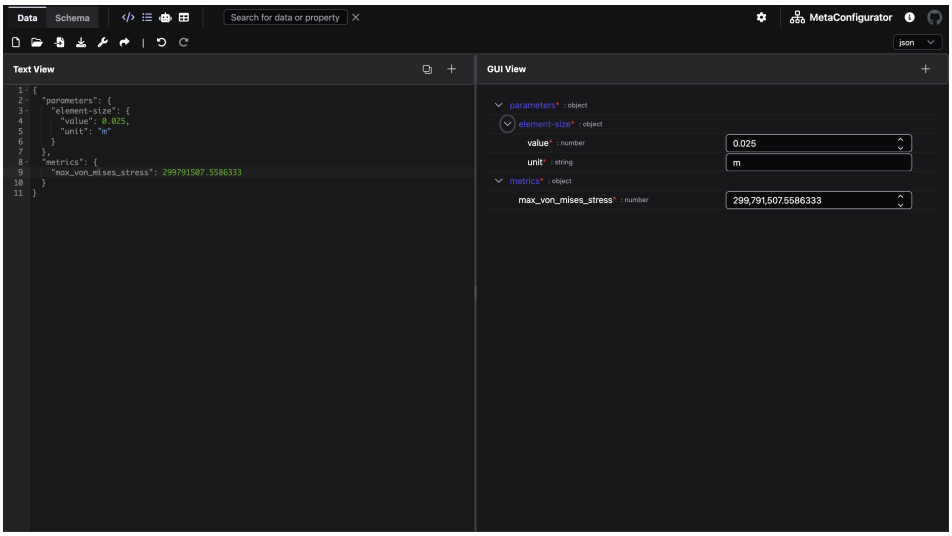


Figure 1.2: A snapshot of MetaConfigurator's data editing interface.

### 1.3.2 AI-Assisted Schema Engineering and Mapping

Recent work extends MetaConfigurator with AI-assisted support for schema creation, schema modification, and schema mapping based on natural-language user input [NUP25]. The approach combines large language models with deterministic safeguards to improve reliability and usability.

For schema authoring, MetaConfigurator provides a chat-like interface where users describe their intent in natural language. The system then performs prompt construction and context management automatically, including role instructions, task-specific formatting constraints, and relevant schema context. Generated results are directly validated and visualized in the schema editor, and users remain in control through a human-in-the-loop workflow where generated output can be reviewed and manually refined before acceptance.

For data transformation, the system generates mapping expressions from user input and executes these mappings deterministically. In the presented implementation, JSONata expressions are generated from an input document and a target schema, then validated and executed in a controlled pipeline. This separates AI-based generation from rule execution and supports scalable transformation workflows across heterogeneous source formats.

### 1.3.3 Limitations for Semantic Web Integration

Despite its strong support for schema-driven data modeling, MetaConfigurator currently provides only limited support for Semantic Web technologies. While the platform enables the creation and validation of structured data using JSON and JSON Schema, it does not natively support Linked Data representations such as JSON-LD or RDF graphs.

As discussed in previous sections, the Semantic Web relies on standards such as RDF, ontologies, and IRIs to represent data in a machine-interpretable and interoperable manner. JSON-LD serves as an important bridge between traditional JSON-based data structures and RDF-based knowledge graphs. However, MetaConfigurator currently treats JSON documents primarily as structured configuration data and does not interpret or expose their underlying RDF semantics.

Several limitations arise from this restriction:

1. **Lack of JSON-LD context authoring:** Users cannot directly author or edit JSON-LD contexts (e.g., the `@context` section that defines mappings between JSON keys and ontology IRIs). Without such support, linking data elements to ontologies requires manual editing outside the tool.

2. **No direct inspection or editing of RDF triples:** MetaConfigurator does not provide mechanisms for inspecting or manipulating RDF triples derived from the underlying JSON data. As a result, users cannot directly view the semantic graph structure implied by JSON-LD documents.

3. **Missing transformation from JSON to RDF:** The platform lacks built-in mechanisms for transforming conventional JSON documents into semantic representations. As discussed in Section 1.2.6, technologies such as RML enable declarative transformations from JSON to RDF, but such transformations are not currently integrated into MetaConfigurator's workflow.

4. **Lack of semantic querying capabilities:** MetaConfigurator does not support semantic querying of RDF data. As described in Section 1.2.4, SPARQL provides a standard mechanism for querying RDF graphs and retrieving structured information from knowledge graphs.

5. **No visualization of RDF knowledge graphs:** Knowledge graphs are often easier to understand when represented visually as nodes and relationships. However, the current platform does not provide mechanisms to visually explore RDF triples or inspect the structure of the resulting knowledge graph.

To address these limitations, this thesis extends MetaConfigurator with an **RDF Authoring Panel** that integrates Semantic Web technologies directly into the schema-driven editing environment. The main contributions of this thesis include:

1. **Transformation of JSON documents to JSON-LD:** The system enables the transformation of conventional JSON documents into JSON-LD using RML mappings. This allows structured JSON data created within MetaConfigurator to be converted into RDF triples that can be integrated into Semantic Web infrastructures.
2. **RDF authoring and editing support:** The RDF Authoring Panel allows users to directly inspect and edit RDF data. This includes editing the JSON-LD `@context` as well as creating and modifying RDF triples within the interface.
3. **Ontology-aware IRI auto-completion:** The system integrates ontology lookup services that provide auto-completion for IRIs when authoring RDF triples. This facilitates the reuse of existing ontologies and promotes semantic interoperability.
4. **SPARQL querying support:** The extension introduces the ability to query RDF data using SPARQL. This allows users to retrieve and analyze information from the generated knowledge graph directly within the MetaConfigurator environment.
5. **Interactive knowledge graph visualization:** The RDF triples generated from JSON-LD can be visualized as an interactive graph representation. This visualization enables users to explore entities and relationships within the knowledge graph and understand the semantic structure of the data more easily.
6. **AI-assisted SPARQL and RML authoring from user hints:** Building on MetaConfigurator's existing AI-assisted schema capabilities, this thesis introduces AI-assisted generation of SPARQL queries and RML mappings from user-provided hints. Users can describe their intent in natural language and receive draft SPARQL queries and RML mapping definitions that can be reviewed and refined in the RDF Authoring Panel.

By integrating these capabilities into MetaConfigurator, the platform evolves from a schema-based configuration editor into a tool that supports the full workflow of creating, transforming, querying, and exploring semantically enriched data.

## 1.4 Related Work

Existing work on Semantic Web tooling can be grouped into four areas: ontology engineering platforms, data-to-RDF transformation tools, schema/metadata modeling environments, and RDF query/processing libraries. Across these areas, strong support exists for individual tasks, but end-to-end workflows that combine schema-driven JSON editing, RDF authoring, mapping, querying, and AI-assisted guidance in one environment remain limited.

### 1.4.1 Ontology Engineering and Knowledge Graph Tooling

Protégé and WebProtégé are widely used for ontology engineering and collaborative ontology curation [CBI; HGN+14]. They provide robust support for class/property modeling and ontology lifecycle management, but they are not primarily designed as schema-driven editors for end-user JSON instance data.

Apache Jena provides a mature programmatic stack for RDF processing, reasoning, and SPARQL querying [Fou]. GraphDB focuses on RDF storage and query execution in production-oriented knowledge graph settings [Ont]. These systems are strong back-end components, but typically require separate front-end layers for guided data entry and authoring workflows.

### 1.4.2 Schema and Metadata Modeling Approaches

LinkML supports schema-first modeling and can generate artifacts such as JSON-LD and Web Ontology Language (OWL) from a single model [MSU+21]. The Center for Expanded Data Annotation and Retrieval (CEDAR) Workbench supports ontology-assisted metadata authoring for scientific experiments [GOM+19]. Both approaches improve semantic consistency at model level, but they do not directly address an integrated workflow where users iteratively transform arbitrary JSON instance data into RDF using declarative mappings and in-tool graph exploration.

### 1.4.3 Data-to-RDF Transformation and Authoring

Karma, OpenRefine (with RDF extensions), and SPARQL-Generate support transformation from heterogeneous sources to RDF [CG; ISI20; Proa]. They are effective for extract, transform, load (ETL)-style conversion tasks, but generally emphasize transformation pipelines over schema-driven configuration editing inside a single application.

### 1.4.4 Querying, Validation, and Lightweight Experimentation

RDF validation and querying standards such as SHACL and SPARQL are well established [PS13; W3C17]. For lightweight experimentation, JSON-LD Playground provides quick inspection of contexts and RDF output [CG23]. RDFLib (RDFLib) and Redland provide low-level programmatic RDF processing [Prob; Proc]. These tools are important ecosystem building blocks but are not, by themselves, unified authoring environments for non-expert users.

### 1.4.5 Comparison of Representative Tools

Table 1.1 compares representative tools across capabilities relevant to this thesis: RDF authoring, ontology integration, data-to-RDF mapping, schema-driven configuration, RDF storage/query, visualization, and AI assistance.

Table 1.1: Comparison of tools supporting RDF authoring and Semantic Web integration

| **Tool / Framework** | RDF Authoring | Ontology Integration | Data-to-RDF Mapping | Schema-Driven Config. | RDF Storage / Query | Visualization | AI Assistance |
|---|---|---|---|---|---|---|---|
| Protégé | ✓ | ✓ | – | – | – | ✓ | – |
| WebProtégé | ✓ | ✓ | – | – | – | Limited | – |
| Apache Jena | ✓ | ✓ | Limited | – | ✓ | Limited | – |
| LinkML | Limited | ✓ | Partial | Partial | – | – | – |
| SemTK | ✓ | ✓ | Partial | – | ✓ | ✓ | – |
| OpenRefine (RDF) | Limited | – | ✓ | – | – | Limited | – |
| SPARQL-Generate | – | Limited | ✓ | – | – | – | – |
| JSON-LD Playground | Limited | – | Limited | – | – | Limited | – |
| RDFLib | ✓ | ✓ | – | – | Partial | – | – |
| Redland RDF | ✓ | ✓ | – | – | Partial | – | – |

### 1.4.6 Research Gap and Positioning of This Thesis

Although the reviewed tools provide strong support for specific Semantic Web tasks, several limitations remain for practical, user-facing scientific data workflows:

- **Fragmented workflows:** Modeling, mapping, querying, and visualization are often distributed across multiple tools, requiring users to switch between different environments to complete a single data integration task [HBC+21].
- **High technical entry barrier:** Effective use often requires advanced knowledge of OWL, RDF, SPARQL, and mapping languages, which limits accessibility for domain scientists who are not Semantic Web experts [JHC+15].

- **Bridging hierarchical data to RDF:** In practice, a large amount of scientific and web data already exists in hierarchical formats such as JSON. While standards such as JSON-LD allow RDF data to be expressed within JSON documents [SKL14b], converting existing hierarchical data models into semantically linked representations typically requires additional transformation languages such as R2RML or RML [DSC12; DVC+14]. However, practical tools that help users incrementally bridge existing JSON-based data to Semantic Web representations remain limited.
- **Limited end-to-end support in one UI:** Most tools focus on a single task—such as ontology editing, mapping definition, or querying—rather than supporting the entire workflow from data authoring and transformation to validation, querying, and graph exploration in a unified environment.
- **Limited AI support in semantic authoring workflows:** Despite recent advances in generative AI, assistance for generating artifacts such as SPARQL queries or RML mappings from natural language or user intent is still rarely integrated into Semantic Web authoring tools.

This thesis addresses these gaps by extending MetaConfigurator from a schema-driven configuration editor into an integrated RDF authoring environment. In particular, it provides a practical bridge from hierarchical JSON-based data models to Semantic Web representations through JSON-to-JSON-LD transformation, ontology-aware authoring support, integrated SPARQL querying, graph visualization, and AI-assisted drafting of SPARQL and RML artifacts.

# 2 Design and Implementation

## 2.1 RDF Panel

## 2.2 RML Mapping Tool

## 2.3 Query with SPARQL

## 2.4 Visualization

# 3 Application

## 3.1 NFDI4ING Model Validation

# 4 Conclusion and Outlook

Conclusion and Outlook

# Bibliography


[BHL01] T. Berners-Lee, J. Hendler, O. Lassila. “The Semantic Web”. In: *Sci. Amer.* 284.5 (2001), pp. 28–37 (cit. on p. 16).

[Bra17] T. Bray. *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC 8259. 2017. URL: https://www.rfc-editor.org/rfc/rfc8259 (cit. on p. 13).

[CBI] S. CBIR. *Protégé*. URL: https://protege.stanford.edu (cit. on p. 28).

[CG] S.-G. CG. *SPARQL-Generate*. URL: https://github.com/SPARQL-Generate (cit. on p. 28).

[CG23] J.-L. CG. *JSON-LD Playground*. 2023 (cit. on p. 28).

[DSC12] S. Das, S. Sundara, R. Cyganiak. *R2RML: RDB to RDF Mapping Language*. https://www.w3.org/TR/r2rml/. W3C Recommendation. 2012 (cit. on p. 30).

[DVC+14] A. Dimou, M. Vander Sande, P. Colpaert, R. Verborgh, E. Mannens, R. Van de Walle. “RML: A Generic Language for Integrated RDF Mappings of Heterogeneous Data”. In: *7th Workshop on Linked Data on the Web*. 2014 (cit. on pp. 20, 30).

[Fou] A. S. Foundation. *Apache Jena*. URL: https://jena.apache.org/ (cit. on p. 28).

[GOM+19] R. S. Gonçalves, M. J. O’Connor, M. Martínez-Romero, et al. “The CEDAR Workbench: An Ontology-Assisted Environment for Authoring Metadata that Describe Scientific Experiments”. In: *arXiv preprint arXiv:1905.06480* (2019) (cit. on p. 28).

[Gru93] T. R. Gruber. “A Translation Approach to Portable Ontology Specifications”. In: *Knowl. Acquis.* 5.2 (1993), pp. 199–220 (cit. on p. 15).

[Gua98] N. Guarino. “Formal Ontology and Information Systems”. In: *Formal Ontology in Information Systems*. IOS Press, 1998, pp. 3–15 (cit. on p. 16).

[HBC+21] A. Hogan, E. Blomqvist, M. Cochez, C. d’Amato, G. de Melo, C. Gutierrez, S. Kirrane, J. E. Labra Gayo, R. Navigli, S. Neumaier, A.-C. N. Ngomo, A. Polleres, S. M. Rashid, A. Rula, L. Schmelzeisen, J. Sequeda, S. Staab, A. Zimmermann. “Knowledge Graphs”. In: *ACM Computing Surveys* 54.4 (2021), pp. 1–37. DOI: 10.1145/3447772 (cit. on p. 29).

[HGN+14] M. Horridge, R. S. Gonçalves, C. Nyulas, T. Tudorache, M. A. Musen. “WebProtégé: A Collaborative Web-Based Platform for Editing Biomedical Ontologies”. In: *Bioinformatics* (2014) (cit. on p. 28).

[ISI20] U. ISI. *Karma: A Data Integration Tool for Mapping Structured Data to RDF*. 2020 (cit. on p. 28).

[JHC+15] K. Janowicz, A. Haller, S. J. D. Cox, D. Le Phuoc, M. Lefrançois. "Why the Data Train Needs Semantic Rails". In: *AI Magazine* 36.1 (2015), pp. 5–14. DOI: 10.1609/aimag.v36i1.2568 (cit. on p. 29).

[MSU+21] S. Moxon, H. Solbrig, D. R. Unni, et al. "Linked Data Modeling Language (LinkML): A General-Purpose Data Modeling Framework". In: *Semantic Web J.* (2021) (cit. on p. 28).

[NBX+24] F. Neubauer, P. Bredl, M. Xu, K. Patel, J. Pleiss, B. Uekermann. "MetaConfigurator: A User-Friendly Tool for Editing Structured Data Files". In: *Datenbank-Spektrum* 24 (2024), pp. 161–169. DOI: 10.1007/s13222-024-00472-7 (cit. on p. 23).

[NUP25] F. Neubauer, B. Uekermann, J. Pleiss. "AI-assisted JSON Schema Creation and Mapping". In: *28th ACM/IEEE Int. Conf. Model Driven Eng. Languages and Systems Companion (MODELS-C)*. 2025, pp. 79–83. DOI: 10.1109/MODELS-C68889.2025.00019 (cit. on p. 25).

[Ont] Ontotext. *GraphDB: RDF Database for Knowledge Graphs*. URL: https://www.ontotext.com/products/graphdb/ (cit. on p. 28).

[Proa] O. Project. *OpenRefine*. URL: https://openrefine.org/ (cit. on p. 28).

[Prob] R. Project. *RDFLib*. URL: https://rdflib.dev/ (cit. on p. 28).

[Proc] R. R. Project. *Redland RDF Libraries*. URL: http://librdf.org/ (cit. on p. 28).

[PS13] E. Prud'hommeaux, A. Seaborne. *SPARQL 1.1 Overview*. 2013. URL: https://www.w3.org/TR/sparql11-overview/ (cit. on pp. 18, 28).

[SBF98] R. Studer, V. R. Benjamins, D. Fensel. "Knowledge Engineering: Principles and Methods". In: *Data & Knowledge Engineering* 25.1–2 (1998), pp. 161–197. DOI: 10.1016/S0169-023X(97)00056-6 (cit. on p. 16).

[SKL14a] M. Sporny, G. Kellogg, M. Lanthaler. *JSON-LD 1.0: A JSON-based Serialization for Linked Data*. 2014. URL: https://www.w3.org/TR/json-ld/ (cit. on p. 19).

[SKL14b] M. Sporny, G. Kellogg, M. Lanthaler. *JSON-LD 1.0: A JSON-based Serialization for Linked Data*. https://www.w3.org/TR/json-ld/. W3C Recommendation. 2014 (cit. on p. 30).

[W3C17] W3C. *Shapes Constraint Language (SHACL)*. 2017. URL: https://www.w3.org/TR/shacl/ (cit. on pp. 17, 28).

[WAHD20] A. Wright, H. Andrews, B. Hutton, G. Dennis. *JSON Schema: A Media Type for Describing JSON Documents*. IETF Draft. 2020. URL: https://json-schema.org/draft/2020-12/ (cit. on p. 14).

All links were last followed on March 10, 2026.

# A Appendix

## A.1 RML Mapping

**Erklärung**

Ich versichere, diese Arbeit selbstständig verfasst zu haben. Ich habe keine anderen als die angegebenen Quellen benutzt und alle wörtlich oder sinngemäß aus anderen Werken übernommene Aussagen als solche gekennzeichnet. Weder diese Arbeit noch wesentliche Teile daraus waren bisher Gegenstand eines anderen Prüfungsverfahrens. Ich habe diese Arbeit bisher weder teilweise noch vollständig veröffentlicht. Das elektronische Exemplar stimmt mit allen eingereichten Exemplaren überein.

---

Ort, Datum, Unterschrift